

MILITARY ASSET MANAGEMENT SYSTEM

Project Documentation & Submission Report

Backend	Django + Django REST Framework
Frontend	React + Vite
Database	PostgreSQL
Authentication	Django authentication + JWT configuration
Environment	Google Colab / Linux runtime for the submitted demo

Purpose

This document covers project scope, architecture, data model, access control, transaction logging, setup, API endpoints and demo credentials.

1. Project Overview

Description

The Military Asset Management System is a web application for tracking military equipment and its movement across multiple bases. It provides a centralized dashboard for purchases, transfers, assignments and expenditures.

Key objectives

- Maintain records of military bases and equipment types.
- Record purchases and increase available inventory.
- Track transfers between source and destination bases.
- Provide visibility into assigned and expended quantities.
- Expose inventory information through REST APIs and a React dashboard.
- Support transaction traceability through structured records and audit logging.

Assumptions

- Each equipment type has a category and unit of measurement.
- Asset quantities are numeric inventory quantities.
- Transfers have source base, destination base, equipment type, quantity, date and status.
- Demo data is illustrative and is not operational military information.
- PostgreSQL is available before the Django backend starts.

Limitations

- Some frontend modules are currently demonstration/summary views rather than full CRUD screens.
- The demo contains purchase and transfer records but no assignment or expenditure records, so those counters can be zero.
- The dashboard endpoint is intentionally public in the demo configuration; production should protect it.
- The current setup is for development/demo execution and needs production hardening.

Demo data: 100 purchased rifle units; 20 transferred in; 20 transferred out. The backend dashboard API returns these values.

2. Tech Stack & Architecture

Layer	Technology	Why chosen
Frontend	React + Vite	Component-based UI and fast development server.
Backend	Python + Django REST Framework	Structured backend, ORM, authentication and REST API development.
Database	PostgreSQL	Relational storage for structured asset and transaction data.
API client	Axios	HTTP communication between React and Django.
Authentication	Django Auth + Simple JWT	User authentication and token-based API support.

Architecture

The application follows a three-layer architecture. React/Vite provides the UI, Django REST Framework provides backend APIs, and PostgreSQL stores persistent records. The frontend calls the backend over HTTP and renders JSON responses.

```
React + Vite → Django REST API → PostgreSQL
React dashboard ← JSON response ← Django
```

The backend is configured for PostgreSQL database **military_assets**. DRF uses JWT authentication and authenticated-by-default permissions; the dashboard endpoint is explicitly public for the current demo.

3. Data Models / Schema

The inventory app uses relational entities for bases, equipment, purchases, transfers, assignments, expenditures and audit records. Django migrations create the corresponding inventory_* tables.

Entity	Purpose	Relationships
Base	Military operating location.	Referenced by purchases and transfer source/destination.
EquipmentType	Defines equipment such as Rifle.	Referenced by purchases, transfers, assignments and expenditures.
Purchase	Records quantity, base, equipment, date and reference number.	Links Base, EquipmentType and created_by User.
Transfer	Records movement between bases, quantity, date and status.	Links from_base, to_base, EquipmentType and created_by User.
Assignment	Tracks equipment assigned to personnel/units.	Links asset/equipment and relevant user/base context.
Expenditure	Tracks consumed or expended assets.	Links equipment and inventory context.
AuditLog	Stores audit/transaction activity.	Supports traceability of users and recorded operations.
Django User	Authentication identity.	Used for login and created_by relationships.

Example flow: Base Alpha → purchase 100 Rifles → transfer 20 Rifles to Base Bravo.

4. RBAC Explanation

The current implementation uses Django's built-in User model and Django REST Framework permissions. There is no separate custom Role model in the current demo; access is based on administrator privileges and authenticated API access.

Access level	Purpose	Enforcement
Administrator / Superuser	System and data administration.	Django <code>is_staff</code> / <code>is_superuser</code> and Django admin permissions.
Authenticated API user	Protected REST API operations.	JWTAuthentication + DRF <code>IsAuthenticated</code> default permission.
Dashboard demo access	View dashboard without token for presentation.	<code>DashboardView</code> sets <code>permission_classes = []</code> .

JWT flow: authenticate → receive access token → send token with API request → DRF authenticates and applies permissions.

Security note: The public dashboard exception is for the submitted demo only and should be removed or replaced by an appropriate permission policy in production.

5. API Logging / Transaction Logging

Transaction traceability is supported by dedicated inventory transaction entities and an AuditLog model/table. Purchases and transfers are stored as structured database records, and created_by associates recorded operations with a Django user where required.

Demo examples

Purchase PO-TEST-001 records 100 Rifle units for Base Alpha. The transfer record records 20 Rifle units from Base Alpha to Base Bravo with status COMPLETED. The dashboard API calculates movement totals from these records.

Audit approach

- Store operational transactions in dedicated relational tables.
- Associate transactions with the responsible authenticated user where required.
- Expose transaction/audit information through backend APIs and the Audit Logs module.
- Use database records as the source of truth for inventory movement.

Current demo limitation: the Audit Logs frontend is a summary/demo view. A production audit screen should show timestamp, user, action, entity, record ID and before/after values where appropriate.

6. Setup Instructions

A. Project structure

```
/military_asset_management/  
■■■ backend/ (Django)  
■■■ frontend/ (React/Vite)  
■■■ database/ (SQL dump)
```

B. Install backend dependencies

```
cd backend  
pip install -r requirements.txt
```

C. Start PostgreSQL and create database

```
sudo apt-get update -qq  
sudo apt-get install -y postgresql postgresql-contrib -qq  
sudo service postgresql start  
sudo -u postgres psql -c "CREATE DATABASE military_assets;"  
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'postgres';"
```

D. Apply migrations

```
python manage.py makemigrations inventory  
python manage.py migrate
```

E. Create demo administrator

Create a Django user named **admin** with password **admin123**, and grant staff/superuser privileges for the demo.

6. Setup Instructions (continued)

F. Run Django backend

```
python manage.py runserver 0.0.0.0:8000
```

G. Run React frontend

```
cd frontend  
npm install  
npm run dev -- --host 0.0.0.0
```

H. Open application

For the Colab demo, expose Vite port 5173 using the Colab port helper. In a normal local setup, open the Vite URL shown by the development server, normally `http://localhost:5173`.

I. Demo data

- Base Alpha — Bengaluru
- Base Bravo — Hyderabad
- Base Charlie — Pune
- Equipment type — Rifle
- Purchase — 100 units at Base Alpha, reference PO-TEST-001
- Transfer — 20 units from Base Alpha to Base Bravo, status COMPLETED

Production note: use environment variables for secrets, restrict CORS, protect sensitive endpoints, use HTTPS and replace the demo password.

7. API Endpoints

The key endpoint used by the submitted dashboard is shown below. Other inventory resource routes may be present in the configured URL patterns.

Method	Endpoint	Purpose	Auth
GET	/api/inventory/dashboard/	Dashboard balances and movement totals.	Public in current demo.
Inventory resource methods	/api/inventory/...	Purchase, transfer, assignment, expenditure and related resources depending on configured routes.	Protected where DRF permissions apply.

Dashboard API sample response

```
{
  "opening_balance": 0,
  "purchases": 100,
  "transfer_in": 20,
  "transfer_out": 20,
  "net_movement": 100,
  "assigned": 0,
  "expended": 0,
  "closing_balance": 100
}
```

The React frontend uses Axios to request this endpoint and render dashboard values.

8. Login Credentials

Working credentials created for the submitted demonstration environment:

Field	Demo value
Username	admin
Password	admin123
Access	Django staff + superuser / administrator-level demo access

Security: These are demonstration credentials. Change the password before any real deployment and keep secrets outside source code.

Submission checklist

- Project overview, assumptions and limitations included.
- Tech stack and architecture documented.
- Core entities and relationships documented.
- RBAC/authentication approach documented.
- Transaction and audit logging approach documented.
- Backend, database and frontend setup documented.
- Key API endpoint and sample response included.
- Working demo login credentials included.

End of document